The C++ Standard Template Library (STL) and File I/O

- The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc. It is a library of container classes, algorithms, and iterators. It is a generalized library and so, its components are parameterized. Working knowledge of template classes is a prerequisite for working with STL.
- The C++ Standard Template Library (STL) is a collection of algorithms, data structures, and other components that can be used to simplify the development of C++ programs. The STL provides a range of containers, such as vectors, lists, and maps, as well as algorithms for searching, sorting and manipulating data.
- One of the key benefits of the STL is that it provides a way to write generic, reusable code that can be applied to different data types. This means that you can write an algorithm once, and then use it with different types of data without having to write separate code for each type.
- The STL also provides a way to write efficient code. Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.

Some of the key components of the STL include:

- **Containers:** The STL provides a range of containers, such as vector, list, map, set, and stack, which can be used to store and manipulate data.
- **Algorithms:** The STL provides a range of algorithms, such as sort, find, and binary_search, which can be used to manipulate data stored in containers.
- **Iterators:** Iterators are objects that provide a way to traverse the elements of a container. The STL provides a range of iterators, such as forward_iterator, bidirectional_iterator, and random_access_iterator, that can be used with different types of containers.
- **Function Objects:** Function objects, also known as functors, are objects that can be used as function arguments to algorithms. They provide a way to pass a function to an algorithm, allowing you to customize its behavior.
- **Adapters:** Adapters are components that modify the behavior of other components in the STL. For example, the reverse_iterator adapter can be used to reverse the order of elements in a container.

- By using the STL, you can simplify your code, reduce the likelihood of errors, and improve the performance of your programs.

STL has 4 components:

- **Algorithms**
- **Containers**
- **Functors**
- **Iterators**

1. Algorithms

- The header algorithm defines a collection of functions specially designed to be used on a range of elements. They act on containers and provide means for various operations for the contents of the containers.
- Algorithm
  - Sorting
  - Searching
  - Important STL Algorithms
  - Useful Array algorithms
  - Partition Operations

- Numeric
  - valarray class
- Containers or container classes store objects and data. There are in total seven standards “first-class” container classes and three container adaptor classes and only seven header files that provide access to these containers or container adaptors.
- Sequence Containers: implement data structures that can be accessed in a sequential manner.
  - vector
  - list
  - deque
  - arrays
  - forward_list(Introduced in C++11)

- Container Adaptors: provide a different interface for sequential containers.
  - queue
  - priority_queue
  - stack
- Associative Containers: implement sorted data structures that can be quickly searched (O(log n) complexity).
  - set
  - multiset
  - map
  - multimap

- Unordered Associative Containers: implement unordered data structures that can be quickly searched
  - unordered_set (Introduced in C++11)
  - unordered_multiset (Introduced in C++11)
  - unordered_map (Introduced in C++11)
  - unordered_multimap (Introduced in C++11)

- **3. Functors**
- The STL includes classes that overload the function call operator. Instances of such classes are called function objects or functors. Functors allow the working of the associated function to be customized with the help of parameters to be passed. **Must Read** – Functors

- **4. Iterators**
- As the name suggests, iterators are used for working on a sequence of values. They are the major feature that allows generality in STL. **Must Read** – Iterators

Utility Library

- C++ includes a variety of utility libraries that provide functionality ranging from bit-counting to partial function application. These libraries can be broadly divided into two groups:
- language support libraries, and
- general-purpose libraries.

Advantages of the C++ Standard Template Library (STL):

- **Reusability**: One of the key advantages of the STL is that it provides a way to write generic, reusable code that can be applied to different data types. This can lead to more efficient and maintainable code.
- **Efficient algorithms:** Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.
- **Improved code readability**: The STL provides a consistent and well-documented way of working with data, which can make your code easier to understand and maintain.
- **Large community of users:** The STL is widely used, which means that there is a large community of developers who can provide support and resources, such as tutorials and forums.

Disadvantages of the C++ Standard Template Library (STL):

- **Learning curve:** The STL can be difficult to learn, especially for beginners, due to its complex syntax and use of advanced features like iterators and function objects.
- **Lack of control**: When using the STL, you have to rely on the implementation provided by the library, which can limit your control over certain aspects of your code.
- **Performance:** In some cases, using the STL can result in slower execution times compared to custom code, especially when dealing with small amounts of data.

Files and Streams

- In C++ programming we are using the **iostream** standard library, it provides **cin** and **cout** methods for reading from input and writing to output respectively.
- To read and write from a file we are using the standard C++ library called **fstream**.

Let us see the data types define in fstream library is:

Data Type	Description
fstream	It is used to create files, write information to files, and read information from files.
ifstream	It is used to read information from files.
ofstream	It is used to create files and write information to the files.

FileStream example: writing to a file

- Let's see the simple example of writing to a text file **testout.txt** using C++ FileStream programming.

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
  ofstream filestream("testout.txt");
  if (filestream.is_open())
  {
    filestream << "Welcome to CSE.\n";
    filestream << "C++ Course.\n";
    filestream.close();
  }
  else cout <<"File opening is fail.";
  return 0;
}
```

Output:

The content of a text file testout.txt is set with the data:

Welcome to CSE.

C++ Course.

C++ FileStream example: reading from a file

- Let's see the simple example of reading from a text file testout.txt using C++ FileStream programming.

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
  string srg;
  ifstream filestream("testout.txt");
  if (filestream.is_open())
  {
    while ( getline (filestream,srg) )
    {
      cout << srg <<endl;
    }
    filestream.close();
  }
  else {
      cout << "File opening is fail."<<endl;
    }
  return 0;
}
```

Note

Note: Before running the code a text file named as "testout.txt" is need to be created and the content of a text file is given below:

Welcome to javaTpoint.

C++ Tutorial.

C++ Read and Write Example

- Let's see the simple example of writing the data to a text file **testout.txt** and then reading the data from the file using C++ FileStream programming.

```
#include <fstream>
#include <iostream>
using namespace std;
int main () {
  char input[75];
  ofstream os;
  os.open("testout.txt");
  cout <<"Writing to a text file:" << endl;
  cout << "Please Enter your name: ";
  cin.getline(input, 100);
  os << input << endl;
  cout << "Please Enter your age: ";
  cin >> input;
  cin.ignore();
  os << input << endl;
  os.close();
  ifstream is;
  string line;
  is.open("testout.txt");
  cout << "Reading from a text file:" << endl;
  while (getline (is,line))
  {
  cout << line << endl;
  }
  is.close();
  return 0;
}
```

Output

Writing to a text file:

Please Enter your name: Nakul Jain

Please Enter your age: 22

Reading from a text file: Nakul Jain

22